CSE 391: System & Software Tools

shell commands, streams, redirection

Ruth Anderson
✉️ rea@cs.uw.edu
🔗 cs.uw.edu/391

Them: How long have you been programming?

Me: Since high school

Them: So you're a good programmer?

Agenda

- Logistics & intro survey recap
- HW1 Q&A
- Brief Pre-lecture Review
  - Useful shell commands (`grep`, `head`, `sort`, `wc`)
  - Streams: standard input, output, error
  - Input/output redirection
  - Pipes
- Discussion Questions
- Tips for HW2

Logistics: Class Structure

If you missed last week...

- this class is semi-flipped (with pre-lectures due before start of class)
- this lecture is recorded (Canvas > Panopto Recordings)
- your entire grade is based on completing **14/20 pts** in assignments
  - No EdStem access? Email me!

Logistics: Getting Help

This course will use Ed as our main Q&A board

- ask clarifying questions about homeworks/lectures/concepts/logistics
- ask code-related questions (if sharing code, **make post private!**)
- trouble accessing any of the course resources (Gradescope, Ed, Canvas)

Anything urgent or wishing to discuss privately

→ please email me at rea@cs.washington.edu

Logistics: Cadence

Overall cadence:

- HW1 was due at 1PM today (before class)
- Either before or soon after class, we'll release:
  - HW2
  - Week 3 Pre-lectures
  - Discussion question solutions
  - Lecture recording
- Sometime this evening (or tomorrow), I'll send an Ed post summarizing all of these!

Logistics: Office Hours

Many many office hours!

- Ruth (me):
  - Mon 1:30 – 2:30pm (CSE 558 & Zoom)
  - Tues 11:00 – 12:00pm (CSE 558 & Zoom)
- Helena (TA):
  - Mon 12:30 – 1:30pm, (CSE2 151)
  - Fri 1:00 – 2:30 pm (CSE2 151)

If none of these times work, please email us and we can figure a time that works does! :)

Logistics: Async Resources

Reference resources:

- Reference sheet
- Lecture notes
- vim/emacs tutorials

Discussion board:

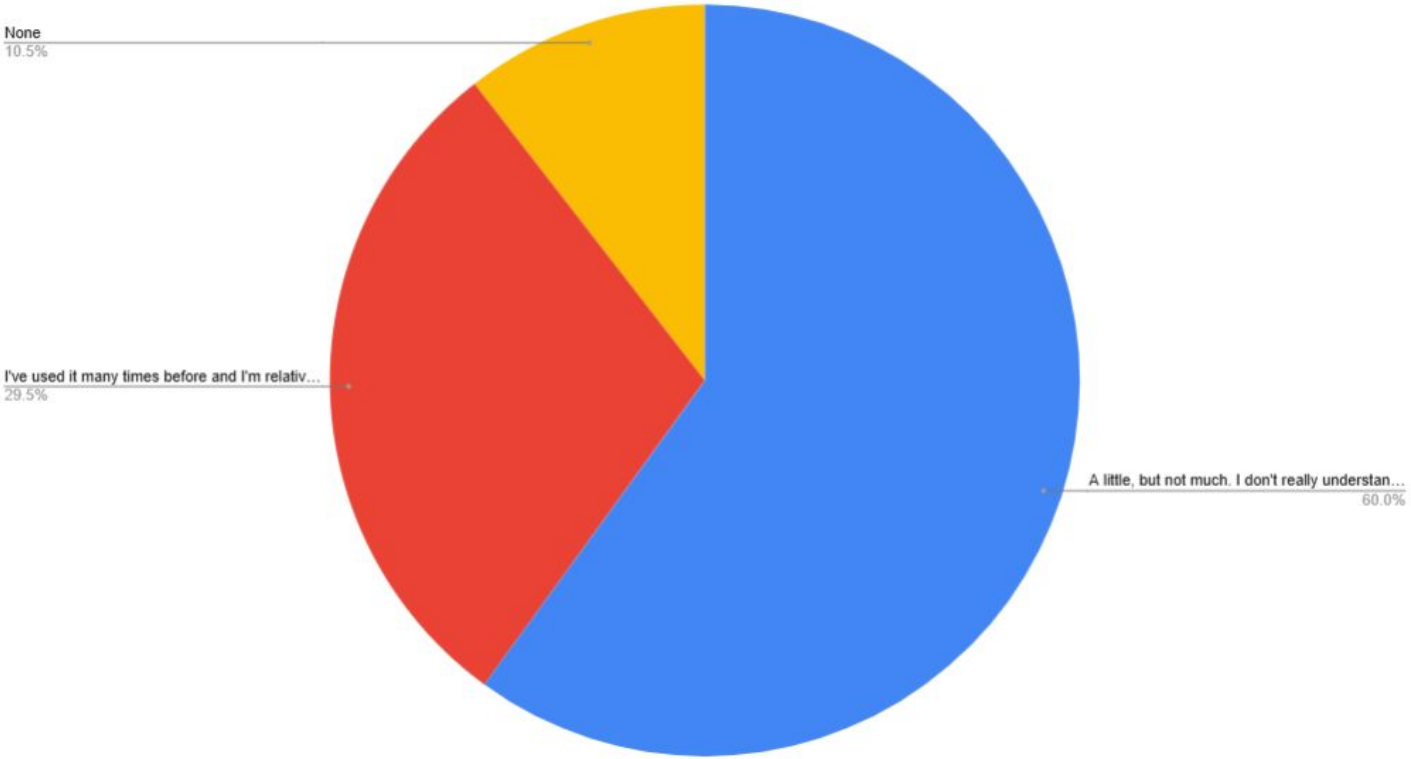
- Keep up the engagement!
- When possible, make posts public so others can learn too
- Think you know the answer? go for it!

A note on homeworks...

Don’t spend too much time grinding on the homeworks!

- Loose rule of thumb: don’t spend more than 15 min “stuck” on a problem
- When stuck,
  - try a different problem
  - ask a question on the discussion board
  - come to office hours
  - take a break (and eat & sleep on it)!

How much experience do you have with git? It's totally fine if your answer is "None" if you haven't heard of git before or never have used it! We will teach it later in this course!



Aside: most of us are beginners!

Live demo: brief pre-lecture review

- Useful shell commands (`wc`, `sort`, `grep`, `uniq`)
- Standard input, output, & error
- Input/output redirection
- Pipes
- How this all fits into the “unix philosophy”

Think, Pair, Share – Skillful Sorting

How would we combine `sort` and `uniq` to remove duplicate lines from `veggies.txt`?

1. Just use `uniq`; `sort` is not necessary
2. Something like: `sort | uniq`
3. Something like: `uniq | sort`
4. Any of these: `sort | uniq` **<u>or</u>** `uniq | sort`

Bonus: what is an example of an actual command to achieve this?

Discussion Questions: Demo

Discussion questions are a chance to bridge the gap between:

- pre-lecture content & review material, “think pair share”
- homeworks (and using it in your future work!)

Works best when you do them!

First: we’ll do Q5, together!

Discussion Questions: Your Turn

In small groups, work on discussion questions (under Week 2)

- prioritize: questions 4, 3, 6
  - at the very least: will do Q3 live
- have a question? we'll be walking around :)
-

HW2 Tips

Biggest tip: solve (bigger) problems in steps!

- Don’t do the entire problem at once; build one command/step at a time
- Also: makes it easier to debug!

Smaller tips:

- **read the hints!**
- Stuck in what seems like an infinite loop? Make sure you’re dealing with input (standard input, arguments, etc.) properly
- Be careful with the header line in the CSV

xz

Semi-recently: a serious backdoor was found in the “xz-utils” (compression) library, maliciously added to the open-source codebase by an unknown user.

It allowed an attacker to bypass security authentication in ssh!

- non-technical piece by the Guardian
- more technical pieces
  - FAQ by Sam James (who also helps maintain Gentoo Linux)
  - Bash obfuscation explanation by Gynvael Coldwind, a reverse engineer
  - Full attack explanation by Russ Cox, security researcher & co-creator of Go at Google

rest: pre-lecture slides (what you saw in the videos)

🤔 Pause and Think (Ticking off TODOs)

You're working on a project and have been leaving comments with the text `"TODO"` near all the things you still need to finish. Your project is stored in the project directory in your current directory.

Write a command to find and print all of the lines that have a TODO on them in the `project` folder.

Hint: Recall for HW1 we taught you how to use the "wildcard" to select all files as part of a path.

```
$ grep "TODO" project/*
```

🤔 Pause and Think (Catch-all Compiling)

What is the command to compile all Java files in the current directory?

Hint: How can you use wildcards to find all Java files?

```
$ javac *.java
```

Standard Streams

Unix processes have three **streams**: abstract locations that tell a program where to read input from and where to write output to.

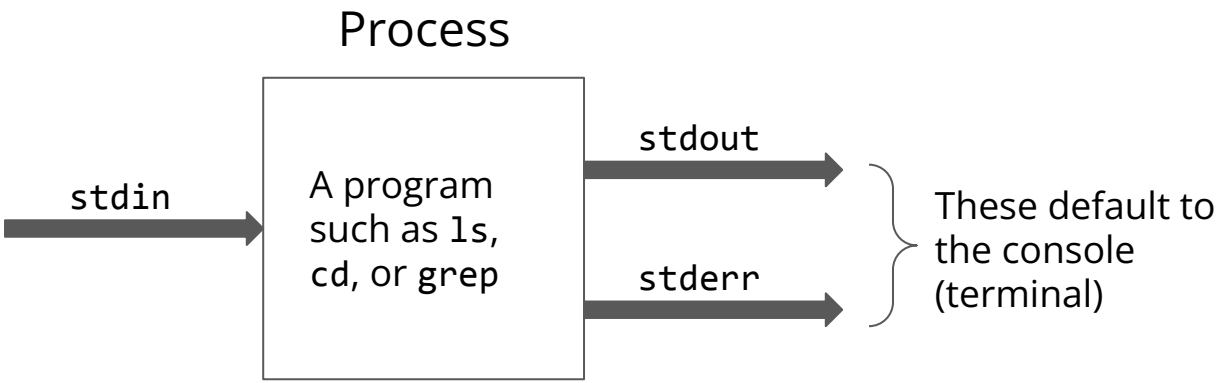
There are three standard streams:

- `stdin` (standard input)
- `stdout` (standard output)
- `stderr` (standard error)

You've implicitly used these when writing Java programs—the `System.out` in `System.out.println` is referring to `stdout`!

Standard Streams Diagram

int	stream	Java
0	stdin	System.in
1	stdout	System.out
2	stderr	System.err



🤔 Pause and Think (Fruit Preserves)

Write a command to store all of the lines in fruits.txt that contain the string `"berry"` into a file called `berries.txt`

```
$ grep "berry" fruits.txt > berries.txt
```

Standard Input versus Parameters

One of the **most important distinctions in this class** is the difference between **standard input** (`stdin`) and a command's **parameters**:

- parameters are arguments given on the command line:
  - e.g. in `ls dir1`, `dir1` is a parameter
  - e.g. in `grep "a"`, `"a"` is a parameter
- standard input comes from the user (either from a file or the console)
  - e.g. in `grep "a" < berries.txt`, the contents of `berries.txt` go into `stdin`
  - e.g. after typing `grep "a"`, it accepts input from your keyboard until the stream is closed (using `Ctrl` + `D`)

🤔 Pause and Think (Bountiful Berries)

Suppose we have a file `berries.txt` where each line is the name of a different berry.

Write a command that outputs how many berries have names that contain **both** the letter **a** and the letter **e**.

```
$ grep "a" berries.txt | grep "e" | wc -l
```

xz & 391

Looking at Gynvael's walkthrough, can we notice some elements of this attack that involve things we've used in this class?

Also:

- we'll learn about some other portions in later weeks (e.g. sed and user groups)
- many interesting related technical & sociotechnical questions
  - are shell scripts & programs easy to read and understand?
  - what does it mean to "trust" software?
  - how do we protect and maintain open-source software?